3. Front-end com react

Sumário

3. Front-end com react 1
MÓDULO 4 – Introdução à Hooks 2
- Aula 1 – O que é um Hook? 2
- Aula 2 – Variáveis de Estado com UseState 2
- Aula 3 – Efeitos Colaterais com UseEffect 6
- Aula 4 – React Router pt. 1: Introdução e Roteamento 7
- Aula 5 – React Router pt. 2: Navegando por uma aplicação com UseNavigate ... 14
- Aula 6 – Gestão de Contexto com UseContext 15
- Fim das Aulas 22

MÓDULO 4 – Introdução à Hooks

➢ Aula 1 – O que é um Hook?

Os Hooks permitem que componentes funcionais acessem o estado e outros recursos do React sem a necessidade de utilizar classes. Com eles, é possível utilizar diferentes funcionalidades do React dentro dos componentes, seja por meio de Hooks integrados ou pela criação de Hooks personalizados.

Neste módulo, iremos aprender os Hooks `useState`, `useContext` e `useEffect`. O `useState` permite manipular o estado da aplicação. O `useContext` possibilita que um componente receba informações de outro componente sem a necessidade de passar dados via props. Já o `useEffect` permite a criação de efeitos colaterais, sendo muito utilizado para conectar e sincronizar o componente com recursos externos, como uma API.

Além disso, vamos aprender a utilizar a biblioteca React Router, que é responsável pelo gerenciamento das rotas da aplicação, bem como o hook `useNavigate`, utilizado para realizar navegação programática entre as páginas.

➢ Aula 2 – Variáveis de Estado com UseState

O `useState` é um Hook amplamente utilizado para controlar quando um componente deve ser renderizado novamente, já que a atualização do estado informa ao React que a interface precisa ser atualizada.

Vamos analisar o que acontece com a atualização do conteúdo da página quando um componente não utiliza o `useState`.

A Imagem 63 ilustra o código do componente. Nele, observamos a existência de uma variável chamada `cont`, inicializada com o valor zero. No JSX retornado pelo componente, há uma tag de parágrafo e um botão, que utiliza o mesmo CSS do componente `Button`. Nesse botão, foi adicionado um evento que, ao ser clicado, chama a função `adicionaUm`, responsável por incrementar o valor de `cont` em uma unidade.

Porém, ao acessarmos o navegador e clicarmos no botão, percebemos que o valor exibido é sempre zero. Independentemente da quantidade de cliques, o valor da variável `cont` exibido na tag `<p>` nunca é atualizado.

Obs.: Dentro do parágrafo, a variável `cont` está entre chaves para indicar que se trata de uma expressão JavaScript. Caso estivesse fora das chaves, o React interpretaria como um texto comum e exibiria a palavra `cont` na tela, em vez do valor armazenado na variável.

```
App.jsx    Counter.jsx ×
src > components > Counter.jsx > Counter > adicionaUm
1   import './Button.css'
2
3   function Counter(props) {
4       var cont = 0
5
6       function adicionaUm() {
7           cont = cont + 1
8       }
9
10
11      return (
12          <>
13              <p>Quantidade de vezes que o usuário clicou no botão: {cont}</p>
14              <button className='button' onClick={adicionaUm}>Clique aqui</button>
15          </>
16      )
17  }
18
19  export default Counter
```

Imagem 63: Componente Counter sem useState

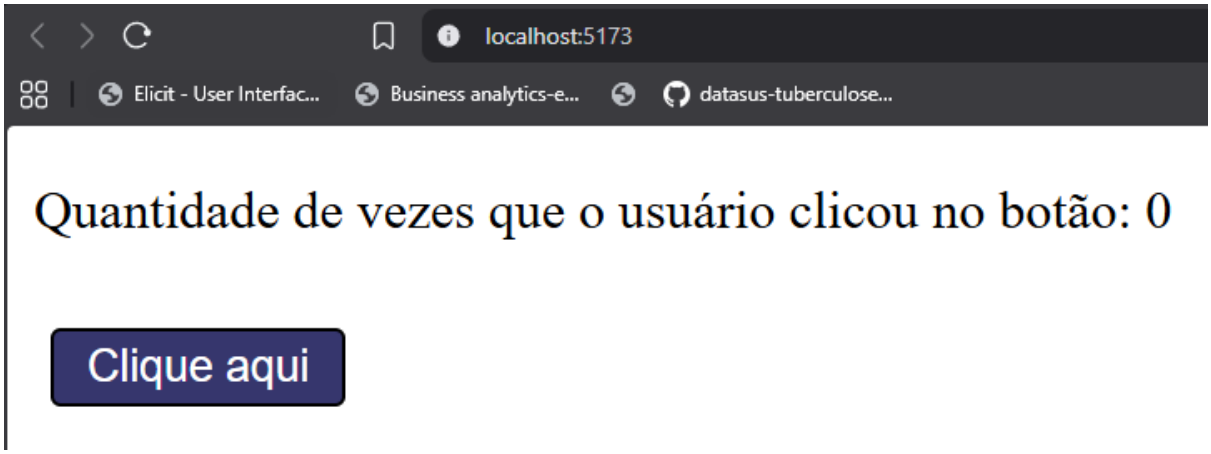


Imagem 64: Resultado no navegador

Podemos então nos perguntar: será que o valor de `cont` realmente não está sendo atualizado?

Para responder a essa pergunta, podemos adicionar um `console.log(cont)` ao final da função `adicionaUm`, conforme ilustrado na Imagem 65. Ao clicar novamente no botão e verificar o console do navegador, observamos o comportamento da variável. Como mostrado na Imagem 66, o valor de `cont` é, de fato, incrementado a cada clique.

```
App.jsx    Counter.jsx ×
src > components > Counter.jsx > Counter > adicionaUm
1   import './Button.css'
2
3   function Counter(props) {
4       var cont = 0
5
6       function adicionaUm() {
7           cont = cont + 1
8           console.log(cont)
9       }
10
11
12      return (
13          <>
14              <p>Quantidade de vezes que o usuário clicou no botão: {cont}</p>
15              <button className='button' onClick={adicionaUm}>Clique aqui</button>
16          </>
17      )
18  }
19
20  export default Counter
```

Imagem 65: Adicionando console.log na função

```
DevTools - localhost:5173/
Elements  Console  Sources  Network  Performance  Memory  Application  Security  Lighthouse  Recorder
top ▾  Filter                                  Default levels ▾  No Issues  2 hidden
                                                                  react-dom_client.js?v=6f25ded8:20101
Download the React DevTools for a better development experience: https://react.dev/link/react-devtools
1                                                                                          Counter.jsx:8
2                                                                                          Counter.jsx:8
3                                                                                          Counter.jsx:8
>|
```

Imagem 66: Console mostrando o valor da variável cont

Então, por que o valor não é atualizado na tela?

Isso acontece porque variáveis comuns no React não têm a capacidade de alterar o estado da aplicação. Sem uma mudança de estado, o React não executa uma nova renderização do componente, e a interface permanece inalterada. Para resolver esse problema, utilizamos o Hook `useState`.

A Imagem 67 ilustra como utilizar o `useState`. Primeiramente, é necessário importá-lo com o seguinte comando:

```
import { useState } from 'react'
```

Em seguida, na sua declaração, devemos observar três pontos importantes: o nome da variável de estado, que neste caso é `cont`; o nome da função responsável por atualizar esse estado, que é `setCont` (por convenção, utiliza-se o prefixo `set` seguido do nome do

estado com a primeira letra maiúscula); e o valor inicial do estado, que neste exemplo é zero e é passado como parâmetro para a função `useState`. Esses pontos estão destacados em vermelho no código apresentado a seguir.

```
const [cont, setCont] = useState(0)
```

Para alterar o valor de `cont`, na função `adicionaUm` utilizamos a função `setCont`, passando como argumento o valor atual do estado somado a uma unidade.

Dessa forma, o valor exibido no parágrafo passa a ser controlado por um `useState`, e não mais por uma variável comum.

A Imagem 68 ilustra o resultado no navegador. Ao clicar no botão, podemos observar que o valor é atualizado corretamente a cada interação.

```
App.jsx    Counter.jsx ×
src > components > Counter.jsx > ...
1   import './Button.css'
2   import { useState } from 'react'
3
4   function Counter(props) {
5       const [cont, setCont] = useState(0)
6
7       function adicionaUm() {
8           setCont(cont + 1)
9       }
10
11
12      return (
13          <>
14              <p>Quantidade de vezes que o usuário clicou no botão: {cont}</p>
15              <button className='button' onClick={adicionaUm}>Clique aqui</button>
16          </>
17      )
18  }
19
20  export default Counter
```

Imagem 67: Componente usando useStante

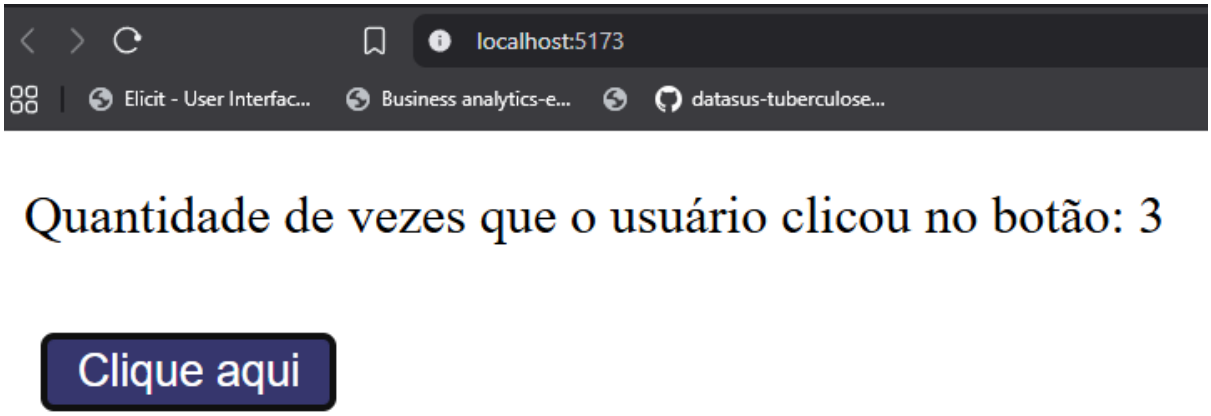


Imagem 68: Resultado no navegador

- **Aula 3 – Efeitos Colaterais com UseEffect**

No React, é comum trabalharmos com funções que fazem mais do que apenas calcular valores. Essas funções podem alterar o estado do componente ou interagir com o mundo externo, como ao realizar requisições HTTP.

Um exemplo disso é a função `buscarUsuarios`, utilizada dentro do `useEffect` na aula de GET do Módulo 3, ilustrada novamente na Imagem 69. Essa função realiza uma requisição GET para obter as informações dos usuários, interagindo com o mundo externo, e em seguida atualiza o estado do componente por meio do `useState`, o que provoca uma nova renderização e, consequentemente, a atualização da página.

Outro conceito importante são os efeitos, que ocorrem sem a necessidade de uma ação direta do usuário. Retomando o exemplo do `useEffect` do GET, ilustrado na Imagem 69, esse efeito é responsável por buscar os dados de todos os usuários assim que o componente é renderizado pela primeira vez. Ou seja, não há um evento como um `onClick` em que o usuário precise clicar em um botão para que as informações sejam carregadas. Nesse caso, o efeito colateral ocorre devido à renderização do componente, e não por uma ação intencional do usuário.

Quando um usuário cria um novo registro utilizando o método POST, temos um evento e não um efeito, pois essa ação depende de uma interação do usuário, como o clique em um botão que dispara um `onClick`. O mesmo acontece com o `onChange` dos inputs, já que o usuário precisa realizar uma ação para que o estado do componente seja alterado.

Em resumo, eventos acontecem em resposta a ações do usuário, enquanto efeitos acontecem em função do ciclo de renderização do componente, podendo ser causados por mudanças de estado, mudanças de props ou pela renderização inicial.

Agora, analisando o código, a Imagem 69 ilustra o uso do `useEffect` para realizar o GET de usuários. O primeiro parâmetro do `useEffect` é a função de efeito que será executada. O segundo parâmetro é a lista de dependências, que pode conter estados ou props.

Sempre que uma dessas dependências mudar e causar uma nova renderização, a função de efeito será executada novamente. No exemplo da Imagem 69, essa lista está vazia, indicando que a função de efeito será executada apenas na primeira renderização do componente.

```
useEffect(() => {
  async function buscarUsuarios() {
    try {
      const resposta = await axios.get("http://localhost:3000/usuarios")
      setUsuarios(resposta.data)
    } catch (error) {
      console.error(error)
    }
  }

  buscarUsuarios()
}, [])
```

Imagem 69: Uso do hook useEffect

- **Aula 4 – React Router pt. 1: Introdução e Roteamento**

O React Router é uma biblioteca utilizada para realizar o roteamento entre as páginas da sua aplicação. Vale observar que as explicações a seguir utilizam o modo declarativo (Declarative Mode) do React Router, mas fique à vontade para usar o Framework Mode ou o Data Mode, todas as três formas de utilização estão disponíveis na documentação oficial.

Para entender melhor o que são rotas, vamos utilizar a URL do site consultasepi.com como exemplo:

Observe a Imagem 70.

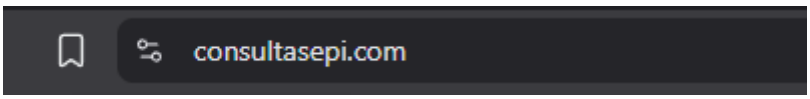


Imagem 70: Url no navegador

Ao entrar no site e observar a URL, vemos que ela é consultasepi.com e que a página inicial do site é exibida. Essa rota é o que chamamos de rota `/`, ela é responsável por mostrar a página inicial, onde o usuário pode começar a navegação na aplicação.

Como ilustrado na Imagem 71, ao observarmos o canto superior direito dessa página, veremos três botões, conforme ilustrado na imagem abaixo.

Imagem 71: Menu do site

O botão que quero destacar é o botão “Ver dashboard”. Ao clicarmos nele, observamos que a URL exibida no navegador muda, como mostrado na Imagem 72.

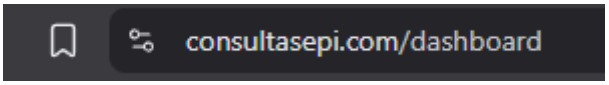


Imagem 72: Url no navegador

Percebemos que agora aparece /dashboard na URL e que o conteúdo do site muda completamente. Com isso, entendemos que estamos em uma segunda página da aplicação.

Para utilizar o React Router, abra o terminal do seu projeto e execute o comando abaixo:

```
npm i react-router
```

Em seguida, vamos envolver a nossa aplicação com o componente `BrowserRouter`, fornecido pelo React Router. Como a aplicação se desenvolve tendo o componente `App` como principal, vamos colocar o `BrowserRouter` envolvendo esse componente no arquivo main.jsx, que é onde o `App` é renderizado.

Para isso, importamos o `BrowserRouter` com o comando abaixo:

```
import { BrowserRouter } from "react-router";
```

Depois, utilizamos esse componente como componente pai do `App`, conforme o exemplo abaixo:

```
 <BrowserRouter>
    <App />
 </BrowserRouter>
```

Observe na Imagem 73 que a importação ocorre na linha 3 e o componente é utilizado nas linhas 8 e 10.

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

Imagem 73: Arquivo main.jsx com o componente BrowserRouter

A partir desse ponto, todas as rotas da aplicação serão definidas no arquivo main.jsx. Para começar, podemos definir a rota principal, que exibirá o conteúdo do componente `App`. Para isso, precisamos definir o `App` como a rota /, conforme ilustrado na Imagem 32.

Primeiro, importamos mais dois componentes: `Routes` e `Route`, conforme mostrado na linha 3 na Imagem 74. Em seguida, utilizando o componente `Route`, definimos o `App` como a rota /. Observe as linhas 9, 10 e 11. O / representa o `path`, ou seja, o caminho da rota que será exibido na URL, e o `element` representa o componente cujo conteúdo será renderizado na tela.

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, Routes, Route } from 'react-router'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<App />} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
)
```

Imagem 74: Uso de Routes e Route

Podemos criar quantas rotas forem necessárias. Na Imagem 75, por exemplo, foram criadas mais duas rotas. A primeira é dedicada à apresentação da empresa, do produto ou do dono do site. A segunda é dedicada às formas de contato, como telefone, e-mail ou qualquer outro meio considerado necessário. Como esse conteúdo é apenas demonstrativo, serão exibidos apenas os títulos desses componentes. Observe as Imagens 75 a 78.

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, Routes, Route } from 'react-router'
import App from './App.jsx'
import About from './components/About.jsx'
import Contact from './components/Contact.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<App />} />
        <Route path='/sobre-nos' element={<About />} />
        <Route path='/contato' element={<Contact />} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
)
```

Imagem 75: Criação de novas rotas

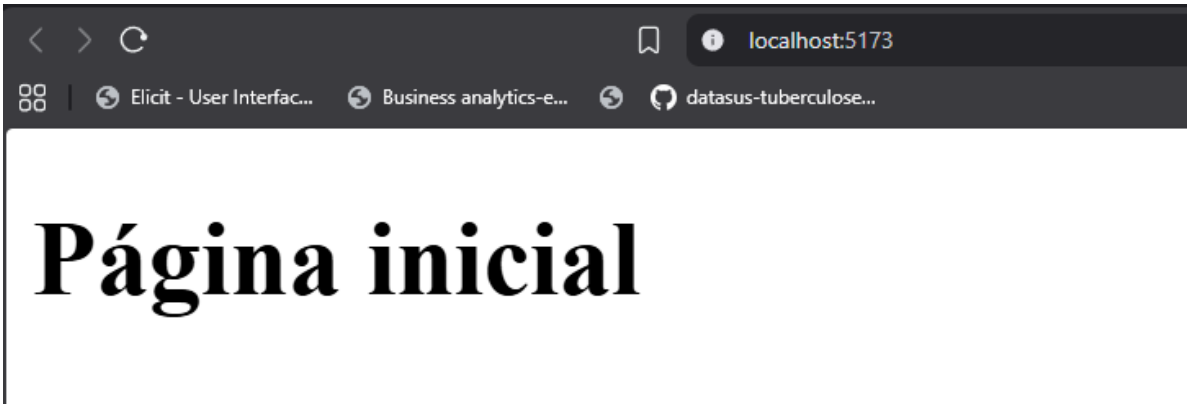


Imagem 76: Conteúdo de App

Ao digitarmos o `path` das outras rotas diretamente na URL do navegador, podemos ver o conteúdo da página mudando completamente, conforme mostrado nas imagens abaixo.

Imagem 77: Conteúdo da página Sobre nós

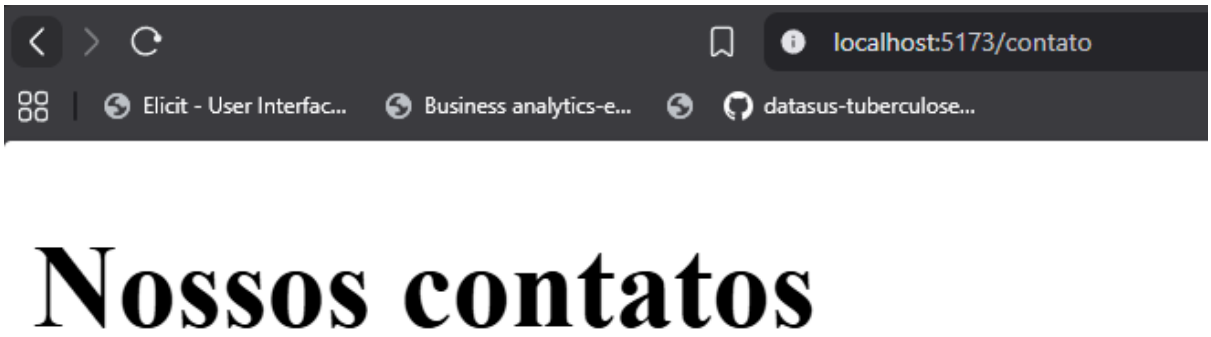


Imagem 78: Conteúdo da página Contatos

Para tornar essa navegação mais simples, sem a necessidade de alterar a URL manualmente, podemos utilizar o componente `Link` do React Router. Ele permite a troca de páginas por meio de cliques. Observe na imagem abaixo a importação e a utilização do componente `Link`, onde o atributo `to` indica para qual rota a aplicação deve navegar. Nesse atributo, utilizamos o mesmo valor definido anteriormente no `path`.

Nas Imagens 79 a 84, é possível ver o conteúdo dos componentes `About` e `Contact`.

```
main.jsx   App.jsx ×   About.jsx   Contact.jsx
src > App.jsx > ...
1   import { Link } from "react-router";
2
3   function App() {
4
5
6     return (
7       <>
8         <h1>Página inicial</h1>
9         <Link to="/sobre-nos">Sobre nós</Link> <br/ >
10        <Link to="/contato">Contato</Link>
11      </>
12    )
13  }
14
15  export default App
16
```

Imagem 79: Conteúdo de App

```
main.jsx ×   App.jsx   About.jsx ×   Contact.jsx
src > components > About.jsx > About
1   import { Link } from "react-router";
2
3   function About() {
4       return (
5           <>
6               <h1>Sobre nós</h1>
7               <Link to="/">Página inicial</Link> <br/ >
8               <Link to="/contato">Contato</Link>
9           </>
10      )
11  }
12
13  export default About
```

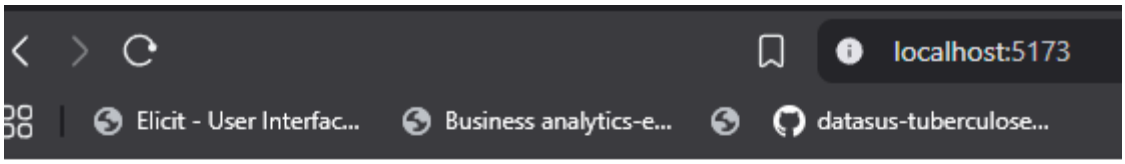
Imagem 80: Conteúdo de sobre nós

main.jsx App.jsx About.jsx Contact.jsx ×

src > components > Contact.jsx > ...

```
1   import { Link } from "react-router";
2
3   function Contact() {
4       return (
5           <>
6               <h1>Nossos contatos</h1>
7               <Link to="/">Página inicial</Link> <br/ >
8               <Link to="/sobre-nos">Sobre nós</Link>
9           </>
10      )
11  }
12
13  export default Contact
14
```

Imagem 81: Conteúdo de contato



Página inicial

Sobre nós
Contato

Imagem 82: Navegador na página inicial

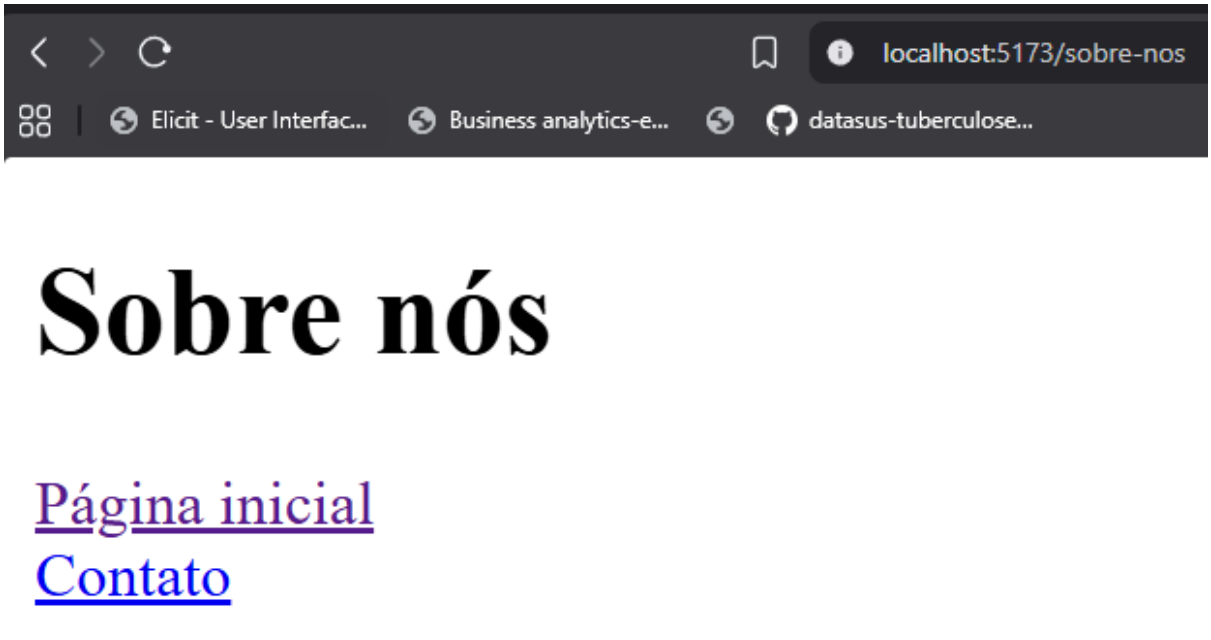


Imagem 83: Navegador na página sobre nós

Imagem 84: Navegador na página contato

Com isso, criamos um menu simples que permite navegar entre as páginas da nossa aplicação de forma prática e organizada.

- **Aula 5 – React Router pt. 2: Navegando por uma aplicação com UseNavigate**

Outra forma de navegar entre as páginas da aplicação é por meio do hook `useNavigate`, disponibilizado pelo React Router. Ele possui a mesma finalidade que o componente `Link`,

com a diferença de que o `Link` é utilizado diretamente no JSX do componente, enquanto o `useNavigate` é utilizado no JavaScript, geralmente dentro de funções.

Para exemplificar o seu uso, vamos alterar um pouco o código presente no componente `App`. Em vez de navegar para a página de contato utilizando o componente `Link`, passaremos a utilizar o hook `useNavigate`.

Para isso, observe a imagem abaixo. Nela, podemos ver a importação da função `useNavigate` na primeira linha, juntamente com o componente `Link`. Em seguida, antes do `return` do componente, utilizamos o `useNavigate`, que retorna uma função. Essa função é armazenada na constante `navigate` e será responsável por realizar a mudança de rotas na aplicação.

Além disso, no retorno JSX, temos um botão que, ao ser clicado, chama a função `irParaContato`. Dentro dessa função, utilizamos a função `navigate`, passando como parâmetro o `path` definido anteriormente para a rota de contato.

Com isso, ao clicarmos no botão, essa navegação será realizada por meio do hook `useNavigate`.

- **Aula 6 – Gestão de Contexto com UseContext**

No React, quando precisamos passar dados de um componente pai para um componente filho, normalmente utilizamos props. No entanto, isso pode se tornar uma tarefa complicada quando uma prop precisa atravessar vários componentes até chegar àquele que realmente necessita do dado. Nesse cenário, muitos componentes intermediários acabam apenas repassando a informação, mesmo sem utilizá-la. Além disso, outro problema surge quando vários componentes diferentes precisam acessar o mesmo dado.

O Context API surge justamente para resolver esse tipo de situação. Ele permite que um dado definido em um componente pai esteja disponível para todos os componentes que estão abaixo dele na árvore, sem a necessidade de passar esse valor manualmente por meio de props.

Em resumo, o contexto funciona como um local central onde o dado é armazenado, enquanto o hook `useContext` é responsável por acessar esse dado em qualquer componente filho que esteja dentro do escopo do contexto. Vale destacar também que é possível alterar o valor do contexto utilizando o hook `useState`, que já foi estudado anteriormente.

Partindo para a prática, vamos iniciar criando um novo projeto, que pode ser chamado de `useContext-inicio`. Em seguida, apague os estilos dos arquivos `index.css` e `App.css` e inicie a aplicação.

Agora, vamos criar o contexto. Para isso, dentro da pasta `src`, crie uma nova pasta chamada `contexts` e, dentro dela, o arquivo `PrimeiroContexto.js`. Nesse arquivo, como ilustrado na Imagem 85, devemos importar a função `createContext` do React. Essa

função recebe como parâmetro o valor inicial do contexto, que pode ser um número, uma string ou até mesmo um objeto com dados mais complexos, como informações de um usuário. Neste exemplo, foi utilizado o valor `1`. Em seguida, atribuímos esse contexto à constante `PrimeiroContexto` e a exportamos, permitindo que ela seja utilizada em outros arquivos do projeto.

```
JS PrimeiroContexto.js ×
src > context > JS PrimeiroContexto.js > PrimeiroContexto
1   import { createContext } from "react";
2
3   export const PrimeiroContexto = createContext(1)
```

Imagem 85: Criando um contexto

Agora que temos o nosso contexto criado, podemos disponibilizá-lo a partir de um componente pai para todos, ou apenas alguns, de seus componentes filhos.

O componente responsável por fornecer o contexto será o `App`. Antes disso, vamos criar alguns componentes auxiliares. Dentro da pasta `src`, crie uma nova pasta chamada `components` e, dentro dela, os componentes `FilhoApp.jsx` e `FilhoFilhoApp.jsx`. As Imagens 86 e 87 ilustram o código inicial desses componentes.

O componente `FilhoApp` importa o componente `FilhoFilhoApp`. Em seu retorno, há uma tag `<h2>` contendo um título e, logo abaixo, a utilização do componente importado.

Por enquanto, o componente `FilhoFilhoApp` possui apenas uma tag `<h3>` com um título.

```
App.jsx ×   FilhoApp.jsx ×   FilhoFilhoApp.jsx   JS
src > components > FilhoApp.jsx > FilhoApp
1   import FilhoFilhoApp from "./FilhoFilhoApp"
2
3   export default function FilhoApp() {
4       return (
5           <>
6               <h2>Filho de App</h2>
7               <FilhoFilhoApp />
8           </>
9       )
10  }
```

Imagem 86: Código de FilhoApp.jsx

```
App.jsx   FilhoApp.jsx   FilhoFilhoApp.jsx
src > components > FilhoFilhoApp.jsx > FilhoFilhoApp
1  export default function FilhoFilhoApp() {
2      return (
3          <>
4              <h3>Filho do Filho de App</h3>
5          </>
6      )
7  }
```

Imagem 87: Código de FilhoFilhoApp.jsx

Com esses componentes criados, podemos voltar ao arquivo `App.jsx` e importar tanto o componente `FilhoApp` quanto o contexto que foi criado anteriormente. A Imagem 88 ilustra essas importações nas linhas 3 e 4.

Além disso, foi criada uma variável de estado chamada `primeiroContexto` utilizando o hook `useState`. É importante não confundir essa variável com o contexto criado anteriormente. O contexto define a estrutura de compartilhamento de dados, enquanto essa variável representa apenas o valor que será fornecido a ele.

```
App.jsx   FilhoApp.jsx   FilhoFilhoApp.jsx   PrimeiroContexto.js
src > App.jsx > App
1  import { useState } from 'react'
2  import './App.css'
3  import FilhoApp from './components/FilhoApp'
4  import { PrimeiroContexto } from './context/PrimeiroContexto'
5
6  function App() {
7    const [primeiroContexto, setPrimeiroContexto] = useState(0)
8
9    return (
10     <>
11
12     </>
13   )
14 }
15
16 export default App
17
```

Imagem 88: Código de App.jsx

Agora, vamos começar a utilizar o componente que foi importado. Observe a linha 12 da Imagem 89, onde o componente `FilhoApp` está sendo renderizado dentro do `App`.

```jsx
import { useState } from 'react'
import './App.css'
import FilhoApp from './components/FilhoApp'
import { PrimeiroContexto } from './context/PrimeiroContexto'

function App() {
  const [primeiroContexto, setPrimeiroContexto] = useState(0)

  return (
    <>
      <h1>App</h1>
      <FilhoApp />
    </>
  )
}

export default App
```

Imagem 89: Componente FilhoApp sendo utilizado em App

A Imagem 90 apresenta o resultado da execução desse código no navegador.

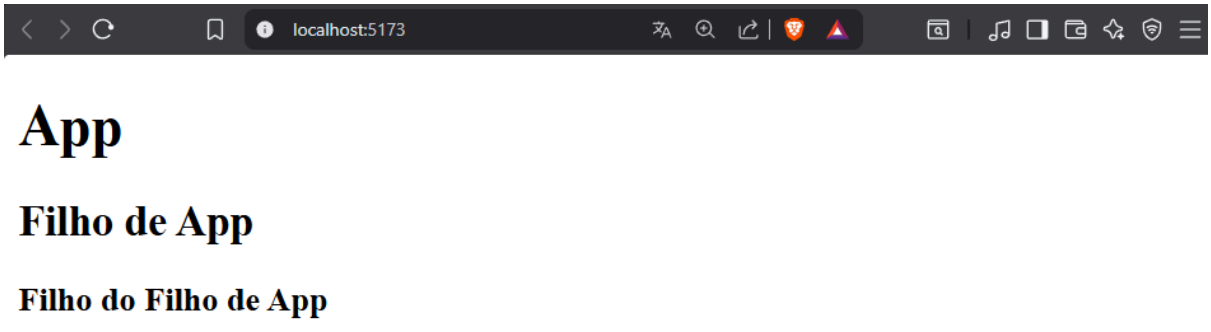


Imagem 90: Resultado no navegador

Agora, vamos disponibilizar o contexto para os componentes filhos de `App`. A Imagem 91 ilustra esse processo. Observe que é necessário envolver, com o provedor do contexto, os componentes que terão acesso a ele. Nesse caso, o `FilhoApp` e todos os componentes que estiverem dentro dele passam a ter acesso ao contexto.

Além disso, por meio da propriedade `value`, passamos a variável responsável por armazenar o valor atual do contexto. Nesse exemplo, utilizamos o estado criado com o hook `useState` na linha 7, que foi inicializado com o valor inicial definido no contexto.

```
App.jsx   FilhoApp.jsx   FilhoFilhoApp.jsx   PrimeiroContexto.js
src > App.jsx > ...
1   import { useState } from 'react'
2   import './App.css'
3   import FilhoApp from './components/FilhoApp'
4   import { PrimeiroContexto } from './context/PrimeiroContexto'
5
6   function App() {
7     const [primeiroContexto, setPrimeiroContexto] = useState(0)
8
9     return (
10      <>
11        <h1>App</h1>
12        <PrimeiroContexto value={primeiroContexto}>
13          <FilhoApp />
14        </PrimeiroContexto>
15      </>
16    )
17  }
18
19  export default App
20
```

Imagem 91: Disponibilizando o contexto

Agora, o valor do contexto está disponível para os componentes `FilhoApp` e `FilhoFilhoApp`, mesmo sem a necessidade de passar qualquer prop entre eles.

Observe a Imagem 92, que apresenta o código do componente `FilhoFilhoApp`. Nesse arquivo, são importados o contexto criado e o hook `useContext`. Na linha 6, podemos ver que o hook `useContext`, ao receber o contexto como parâmetro, permite atribuir à constante `primeiroContexto` o valor atual do contexto.

Além disso, foi adicionada uma tag `<p>` abaixo do título `<h3>` com o objetivo de exibir o valor do contexto na tela.

A Imagem 93 ilustra o resultado dessa implementação no navegador.

```
App.jsx    FilhoApp.jsx    FilhoFilhoApp.jsx ×    PrimeiroContexto.js
src > components > FilhoFilhoApp.jsx > ...
1   import { PrimeiroContexto } from "../context/PrimeiroContexto"
2   import { useContext } from "react"
3
4   export default function FilhoFilhoApp() {
5
6       const primeiroContexto = useContext(PrimeiroContexto)
7
8       return (
9           <>
10              <h3>Filho do Filho de App</h3>
11              <p>Valor do contexto: {primeiroContexto}</p>
12          </>
13      )
14  }
15
```

Imagem 92: Utilizando o valor do contexto

localhost:5173

App

Filho de App

Filho do Filho de App

Valor do contexto: 0

Imagem 93: Resultado no navegador

Mas e se criarmos um botão que altere o valor da variável de estado `primeiroContexto` no componente `App`, o valor do contexto em `FilhoFilhoApp` será atualizado?

Observe a linha 12 da Imagem 94, onde foi adicionado um botão responsável por incrementar em `1` o valor do estado `primeiroContexto`, definido com o hook `useState`.

```jsx
import { useState } from 'react'
import './App.css'
import FilhoApp from './components/FilhoApp'
import { PrimeiroContexto } from './context/PrimeiroContexto'

function App() {
  const [primeiroContexto, setPrimeiroContexto] = useState(0)

  return (
    <>
      <h1>App</h1>
      <button onClick={() => setPrimeiroContexto((prev) => prev + 1)}>Adicione 1</button>
      <PrimeiroContexto value={primeiroContexto}>
        <FilhoApp />
      </PrimeiroContexto>
    </>
  )
}

export default App
```

Imagem 94: Botão para atualizar o valor do contexto

Observe o resultado no navegador, ilustrado na Imagem 95. Ao clicar no botão, o valor do contexto é atualizado automaticamente.

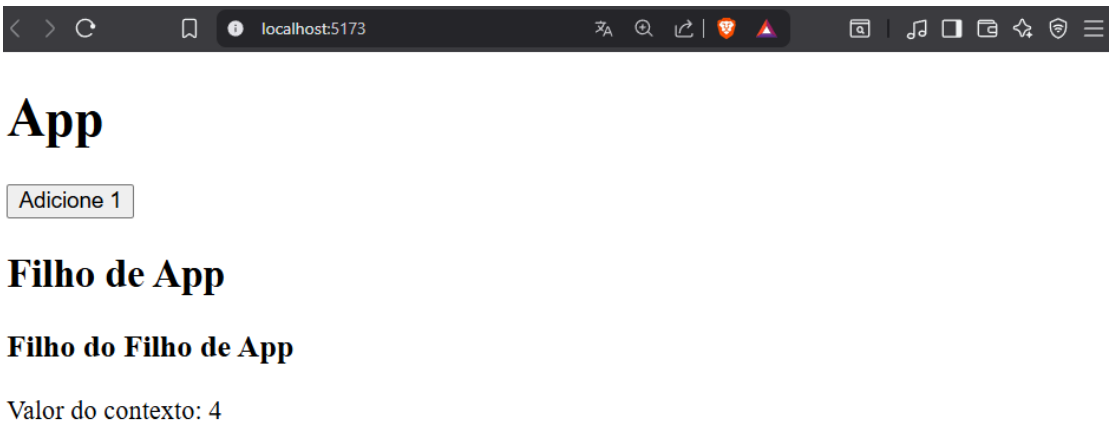


Imagem 95: Resultado no navegador

Com isso, percebemos que, mesmo com o valor sendo atualizado em um componente diferente daquele que o exibe, e sem a necessidade de passar props entre eles, a atualização ocorre normalmente. Isso acontece porque ambos os componentes têm acesso ao mesmo contexto.

- **Fim das Aulas**

Chegamos ao final das aulas de React. Esperamos que, ao longo dessa jornada, você tenha adquirido conhecimentos importantes e desenvolvido uma boa base para continuar evoluindo na área.

Agora é o momento de colocar tudo em prática. Utilize o que foi aprendido para criar seus próprios projetos, explorar novas ideias e enfrentar desafios cada vez mais complexos. A prática constante é fundamental para o seu crescimento.

Continue estudando, buscando novos conteúdos e se aprofundando cada vez mais. O aprendizado não termina aqui, ele é um processo contínuo.

Desejamos muito sucesso na sua trajetória e que você siga evoluindo cada vez mais como desenvolvedor.